UNIVERSIDAD DON BOSCO



ASIGNATURA: PROGRAMACIÓN ORIENTADA A OBJETOS POO901 G01T

EXPERIENCIA DE APRENDIZAJE 3

FASE 1: SISTEMA DESKTOP BASE CON SWING, POO, MYSQL Y JDBC

GRUPO DE TRABAJO NÚMERO CUATRO

INTEGRANTES:

CRISTIAN ALEXANDER CAMPOS DIAZ - CD261750

SAMUEL OSWALDO CHICAS COLORADO - CC220538

CRYSTEL DANIELA TINO ANAYA - TA261223

LESTER DANIEL HURTADO HURTADO - HH261245

LUIS ALBERTO CHÁVEZ MENDOZA – CM250419

CICLO II 2026

SÁBADO 7 DE NOVIEMBRE DE 2026

1.Introducción.

El presente proyecto consiste en el desarrollo de un sistema de gestión bancaria para el Banco Académico UDB, cuyo propósito es facilitar la administración de clientes y cuentas bancarias mediante una aplicación de escritorio. Esta solución busca mejorar la organización de la información, reducir los errores que pueden producirse durante el registro manual de datos y evitar la duplicidad de registros, proporcionando una herramienta que permita gestionar la información de manera ordenada y eficiente.

Para el desarrollo de esta primera fase se utilizará el lenguaje de programación Java, implementando interfaces gráficas mediante la biblioteca Swing y aplicando los principios de la Programación Orientada a Objetos (POO) para organizar y estructurar el código. Asimismo, se utilizará MySQL como sistema de gestión de bases de datos para almacenar la información de forma persistente, mientras que JDBC permitirá establecer la comunicación entre la aplicación Java y la base de datos.

La Fase 1 se centra en la construcción de la estructura base del sistema, incorporando la gestión de clientes mediante operaciones de registro, consulta, modificación y eliminación (CRUD), así como la creación y administración de cuentas bancarias. También se contempla la implementación de validaciones para garantizar que los datos ingresados cumplan con los criterios establecidos.

Finalmente, este documento técnico presenta la estructura, las tecnologías, el diseño y las funcionalidades desarrolladas durante esta fase, además de las validaciones, los casos de prueba y las evidencias de ejecución que permiten comprobar el funcionamiento de la aplicación y el cumplimiento de los objetivos establecidos.

2. Planteamiento del problema.

El Banco Académico UDB presenta la necesidad de mejorar la administración y organización de la información relacionada con sus clientes y cuentas bancarias. Actualmente, el manejo manual de estos registros puede ocasionar inconsistencias en los datos, duplicidad de cuentas y dificultades al momento de consultar la información, lo que afecta el orden y la eficiencia de los procesos administrativos.

La ausencia de una aplicación centralizada para gestionar estos registros dificulta mantener la información organizada y realizar operaciones de consulta, actualización y eliminación de manera sistemática. Asimismo, la falta de controles durante el registro puede provocar errores en los datos proporcionados por los clientes, generando inconvenientes en su almacenamiento y posterior consulta.

Ante esta situación, surge la necesidad de desarrollar una aplicación de escritorio que permita administrar de manera estructurada la información de los clientes y sus respectivas cuentas bancarias. El sistema deberá facilitar el registro, consulta, modificación y eliminación de clientes, además de incorporar mecanismos de validación que contribuyan a mantener la integridad y consistencia de la información almacenada.

Por lo tanto, el desarrollo de esta primera fase busca proporcionar una solución informática mediante Java, Swing, Programación Orientada a Objetos, MySQL y JDBC, que permita establecer una base funcional para la gestión de clientes y cuentas bancarias del Banco Académico UDB.

3. Justificación

El desarrollo del Sistema de Gestión Bancaria Académica se justifica por la necesidad de mejorar la organización, administración y almacenamiento de la información relacionada con los clientes y sus cuentas bancarias. La implementación de una aplicación de escritorio permitirá sustituir el manejo manual de los registros por un sistema centralizado que facilite la gestión de los datos, reduzca la posibilidad de errores y contribuya a evitar la duplicidad de información.

Asimismo, el sistema permitirá realizar operaciones de registro, consulta, modificación y eliminación de clientes mediante una interfaz gráfica que facilite la interacción con el usuario. La incorporación de validaciones contribuirá a garantizar que los datos ingresados cumplan con los formatos y requisitos establecidos, mientras que el almacenamiento mediante una base de datos MySQL permitirá conservar la información para su posterior consulta y administración.

Desde el punto de vista tecnológico y académico, este proyecto representa una oportunidad para aplicar los conocimientos adquiridos sobre Programación Orientada a Objetos, mediante el desarrollo de clases, objetos y estructuras que favorezcan la organización y reutilización del código. Además, el uso de Java Swing permitirá construir una interfaz gráfica de escritorio, mientras que JDBC facilitará la comunicación entre la aplicación y la base de datos.

Finalmente, esta primera fase permitirá establecer una estructura funcional para el Sistema de Gestión Bancaria Académica, sentando las bases para su posterior ampliación en las siguientes etapas del proyecto. De esta manera, se busca obtener una aplicación organizada, funcional y adaptable a los requerimientos establecidos para la administración de clientes y cuentas bancarias.

4. Objetivos generales y específicos

4.1. Objetivo general

Desarrollar una aplicación de escritorio para el Banco Académico UDB utilizando el lenguaje de programación Java, la biblioteca gráfica Swing, los principios de la Programación Orientada a Objetos (POO), MySQL y JDBC, que permita administrar de manera organizada la información de los clientes y sus respectivas cuentas bancarias.

4.2. Objetivos específicos

1. Implementar un módulo de gestión de clientes que permita realizar operaciones de registro, consulta, modificación y eliminación de información mediante las funcionalidades CRUD.
2. Desarrollar la estructura necesaria para la creación y administración de cuentas bancarias asociadas a los clientes registrados en el sistema.
3. Desarrollar una interfaz gráfica de escritorio utilizando Java Swing que facilite la interacción del usuario con las diferentes funcionalidades del sistema.

4. Diseñar e implementar una base de datos relacional en MySQL que permita almacenar y organizar la información de los clientes y sus respectivas cuentas bancarias.
5. Establecer la conexión entre la aplicación Java y la base de datos MySQL mediante JDBC, permitiendo realizar operaciones de consulta y manipulación de información.
6. Aplicar los principios de la Programación Orientada a Objetos, incluyendo clases, objetos, encapsulamiento y herencia, para desarrollar una estructura de código organizada y reutilizable.
7. Incorporar validaciones que permitan verificar la información ingresada por los usuarios, evitando registros incompletos, datos incorrectos y duplicidad de información.
8. Realizar pruebas de funcionamiento que permitan comprobar las operaciones del sistema, la conexión con la base de datos y el cumplimiento de los requisitos establecidos para esta primera fase.

5. Alcance de la Fase 1

5.1. Alcance funcional

La primera fase del proyecto contempla el desarrollo de una aplicación de escritorio destinada a la gestión de clientes y cuentas bancarias del Banco Académico UDB.

El sistema incluirá un módulo de gestión de clientes que permitirá realizar las operaciones básicas de registro, consulta, modificación y eliminación de información. Cada cliente contará con un registro que contendrá los siguientes datos: código de cliente, DUI, nombre completo, teléfono, correo electrónico, dirección y estado, el cual podrá ser Activo, Inactivo o Bloqueado.

Asimismo, se contempla la creación y administración de cuentas bancarias, permitiendo establecer la estructura necesaria para relacionarlas con los clientes registrados en el sistema.

La aplicación contará con una interfaz gráfica desarrollada mediante Java Swing, facilitando la interacción del usuario con las funcionalidades disponibles. También incorporará mecanismos de validación para verificar los datos ingresados y una conexión con la base de datos MySQL mediante JDBC, garantizando el almacenamiento y la consulta de la información.

5.2. Alcance tecnológico

Para el desarrollo de esta fase se utilizarán las siguientes tecnologías:

- **Java:** lenguaje de programación utilizado para implementar la lógica y las funcionalidades de la aplicación.
- **Java Swing:** biblioteca utilizada para construir las interfaces gráficas de la aplicación de escritorio.
- **Programación Orientada a Objetos:** paradigma empleado para organizar el código mediante clases y objetos, aplicando principios como encapsulamiento y herencia.
- **MySQL:** sistema de gestión de bases de datos encargado de almacenar la información de los clientes y las cuentas bancarias.
- **JDBC:** tecnología utilizada para establecer la comunicación entre la aplicación Java y la base de datos MySQL.

Además, se desarrollará el script SQL necesario para la creación de la estructura de la base de datos.

5.3. Límites de la Fase 1

Esta primera etapa se concentra en establecer la estructura base del sistema bancario mediante la gestión de clientes y cuentas bancarias. Por lo tanto, no contempla todavía la implementación de operaciones bancarias avanzadas ni la migración de la aplicación a un entorno web.

Las funcionalidades de depósitos, retiros, transferencias, historial de movimientos y generación de reportes administrativos corresponden a la segunda fase del proyecto. De igual manera, el desarrollo de una aplicación web mediante HTML, Servlets, JSP y JavaBeans será abordado en la tercera fase.

De esta manera, el alcance de la Fase 1 se limita a construir una aplicación de escritorio funcional que permita administrar clientes y cuentas bancarias, estableciendo una base tecnológica y estructural para las siguientes etapas del proyecto.

6. Tecnologías utilizadas

Para el desarrollo del Sistema de Gestión Bancaria Académica se emplearán diferentes tecnologías que permitirán construir la aplicación de escritorio, organizar su estructura interna y gestionar el almacenamiento de la información. Cada una cumplirá una función específica dentro del proyecto, trabajando de manera conjunta para cumplir con los requerimientos establecidos para la primera fase.

6.1. Java

Java es el lenguaje de programación utilizado para desarrollar la lógica y las funcionalidades principales del sistema. Mediante este lenguaje se implementarán las clases y los métodos necesarios para representar a los clientes y las cuentas bancarias, así como las operaciones relacionadas con su administración.

Su utilización permitirá organizar el código mediante el enfoque de Programación Orientada a Objetos, facilitando la estructuración y el mantenimiento de la aplicación.

6.2. Java Swing

Java Swing es una biblioteca de Java que permite desarrollar interfaces gráficas de usuario para aplicaciones de escritorio. Proporciona componentes como ventanas, botones, etiquetas, campos de texto, tablas y cuadros de diálogo, que facilitan la interacción entre el usuario y el sistema (Oracle, s. f.).

En este proyecto, Swing se utilizará para construir las ventanas y formularios desde los cuales se podrán realizar las operaciones de gestión de clientes y cuentas bancarias. De esta manera, el usuario podrá interactuar con las funcionalidades de la aplicación mediante una interfaz gráfica.

6.3. Programación Orientada a Objetos (POO)

La Programación Orientada a Objetos es un paradigma de programación que permite organizar el código mediante clases y objetos, agrupando datos y comportamientos relacionados. Entre sus principios se encuentran el encapsulamiento y la herencia, los cuales contribuyen a estructurar y reutilizar el código.

Dentro del proyecto, este paradigma se aplicará mediante la creación de clases que representen los elementos principales del sistema, como las personas, los clientes y las cuentas bancarias. También permitirá establecer relaciones entre las clases y organizar las responsabilidades de cada una, procurando mantener una estructura clara y ordenada.

6.4. MySQL

MySQL es un sistema de gestión de bases de datos que permitirá almacenar y administrar la información utilizada por la aplicación. Su función dentro del proyecto será mantener los registros de clientes y cuentas bancarias de manera persistente, permitiendo que los datos permanezcan almacenados y puedan consultarse posteriormente.

La base de datos se organizará mediante tablas y relaciones que permitan estructurar la información de acuerdo con los requerimientos establecidos para esta primera fase. Asimismo, se utilizarán instrucciones SQL para la creación de la base de datos y la manipulación de los registros (Oracle, 2026; MySQL, s. f.).

6.5. JDBC (Java Database Connectivity)

JDBC es una tecnología de Java que permite establecer la comunicación entre una aplicación y una base de datos mediante una interfaz estándar de conectividad. A través de esta tecnología, una aplicación puede establecer conexiones, ejecutar instrucciones SQL y recuperar los resultados de las consultas realizadas (Oracle, s. f.).

En el Sistema de Gestión Bancaria Académica, JDBC se utilizará para conectar la aplicación desarrollada en Java con la base de datos MySQL. Esto permitirá realizar las operaciones necesarias para registrar, consultar, modificar y eliminar información almacenada.

Para establecer esta comunicación con MySQL se utilizará el controlador MySQL Connector/J, que implementa la API JDBC y permite que las aplicaciones desarrolladas en Java interactúen con el servidor de bases de datos (MySQL, s. f.).

6.6. Script SQL

El script SQL es un archivo que contiene las instrucciones necesarias para crear y configurar la estructura de la base de datos. En este proyecto se elaborará para definir la base de datos y las tablas correspondientes a la gestión de clientes y cuentas bancarias.

Este archivo permitirá disponer de una estructura de base de datos que pueda crearse y configurarse en el entorno MySQL, facilitando la preparación del sistema para su ejecución.

7. Arquitectura del sistema

El Sistema de Gestión Bancaria Académica está desarrollado mediante una aplicación de escritorio en Java, utilizando Swing para la interfaz gráfica, programación orientada a objetos para la organización de sus componentes y JDBC para la comunicación con MySQL.

La estructura del proyecto se organiza en paquetes que permiten separar las responsabilidades de la aplicación:

Paquete sistema: contiene las clases que representan las entidades principales del sistema, como Persona, Cliente y Cuenta_bancaria, además de la clase Main, encargada de iniciar la aplicación.

Paquete vistas: contiene las interfaces gráficas desarrolladas con Swing. Incluye el menú principal y los módulos para la gestión de clientes y cuentas bancarias.

Paquete dao: contiene las clases ClientesDao y CuentasDao, responsables de ejecutar las operaciones de consulta, registro, actualización y eliminación de información en la base de datos.

Paquete conexion: contiene la clase Conexion, encargada de establecer las conexiones con MySQL mediante JDBC.

Archivos de configuración y base de datos: fuera de los paquetes Java se encuentran las carpetas config_database y database. La primera contiene el archivo database.properties, utilizado para almacenar los parámetros de conexión con MySQL. La segunda contiene el script SQL utilizado para la creación de la base de datos.

Esta organización permite separar la interfaz gráfica, la representación de los datos, las operaciones de persistencia y la conexión con la base de datos, facilitando el mantenimiento y la comprensión del sistema.

8. Diseño y estructura de la base de datos

8.1. Nombre de la base de datos

Para el desarrollo del Sistema de Gestión Bancaria Académica se creó una base de datos relacional denominada sistema_bancario_udb, utilizando MySQL. Su propósito es almacenar y organizar la información de los clientes y sus respectivas cuentas bancarias, garantizando la integridad y consistencia de los datos.

La base de datos está compuesta por dos tablas principales: clientes y cuentas_bancarias.

Evidencia 1. Estructura general de la base de datos.

Figura 1. *Estructura general de la base de datos sistema_bancario_udb.*



Nota. Elaboración propia mediante phpMyAdmin.

8.2. Descripción de las tablas

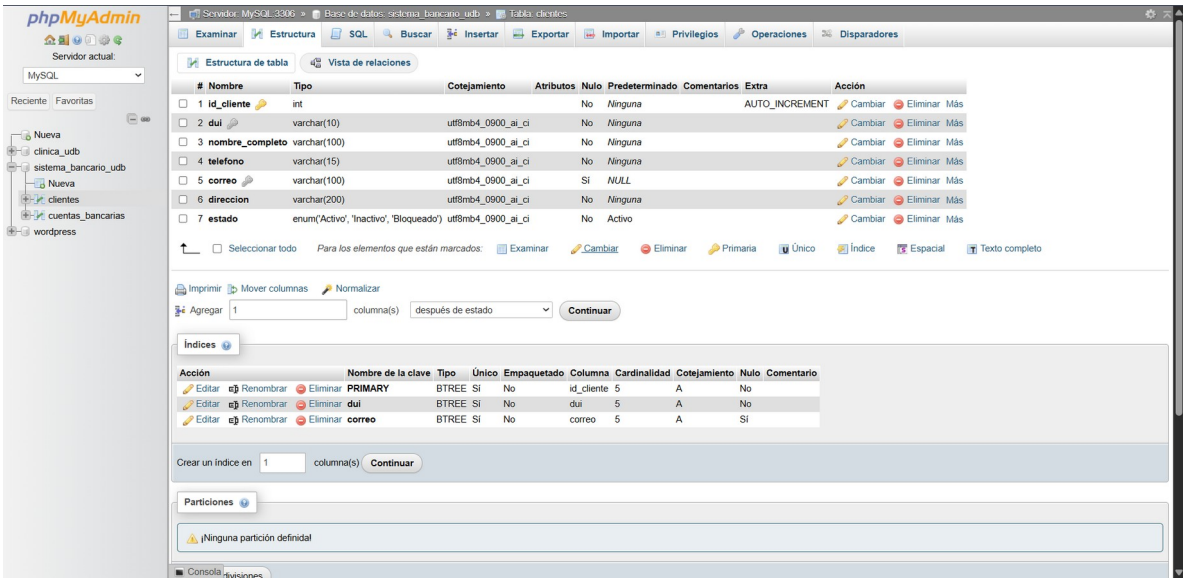
Tabla CLIENTES

Esta tabla almacena la información personal y administrativa de los clientes registrados en el sistema.

Campo	Tipo de dato	Descripción
id_cliente	INT	Identificador único del cliente.
dui	VARCHAR(10)	Documento Único de Identidad.
nombre_completo	VARCHAR(100)	Nombre completo del cliente.
telefono	VARCHAR(15)	Número telefónico.
correo	VARCHAR(100)	Correo electrónico, opcional.
direccion	VARCHAR(200)	Dirección del cliente.
estado	ENUM	Estado: Activo, Inactivo o Bloqueado.

El campo id_cliente funciona como clave primaria y utiliza AUTO_INCREMENT. El DUI y el correo electrónico poseen restricciones de unicidad para evitar registros duplicados. El correo permite valores nulos y el estado tiene como valor predeterminado «Activo».

Figura 2. *Estructura de la tabla clientes.*



Nota. Elaboración propia mediante phpMyAdmin.

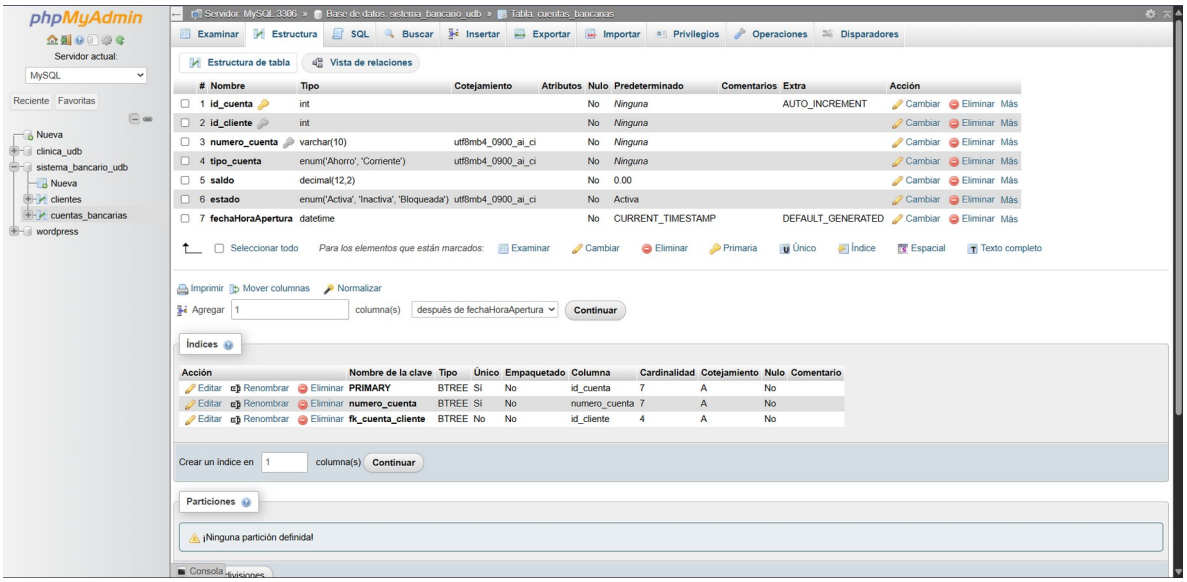
Tabla CUENTAS_BANCARIAS

Esta tabla almacena la información correspondiente a las cuentas bancarias registradas y su relación con los clientes.

Campo	Tipo de dato	Descripción
id_cuenta	INT	Identificador único de la cuenta.
id_cliente	INT	Identificador del cliente propietario.
numero_cuenta	VARCHAR(10)	Número único de la cuenta bancaria.
tipo_cuenta	ENUM	Tipo de cuenta: Ahorro o Corriente.
saldo	DECIMAL(12,2)	Saldo disponible en la cuenta.
estado	ENUM	Estado: Activa, Inactiva o Bloqueada.
fechaHoraApertura	DATETIME	Fecha y hora de apertura de la cuenta.

El campo id_cuenta es la clave primaria y utiliza AUTO_INCREMENT. El número de cuenta posee una restricción de unicidad. El saldo tiene un valor predeterminado de 0.00, mientras que el estado se establece inicialmente como «Activa» y la fecha de apertura se genera mediante CURRENT_TIMESTAMP.

Figura 3. *Estructura de la tabla cuentas_bancarias*.



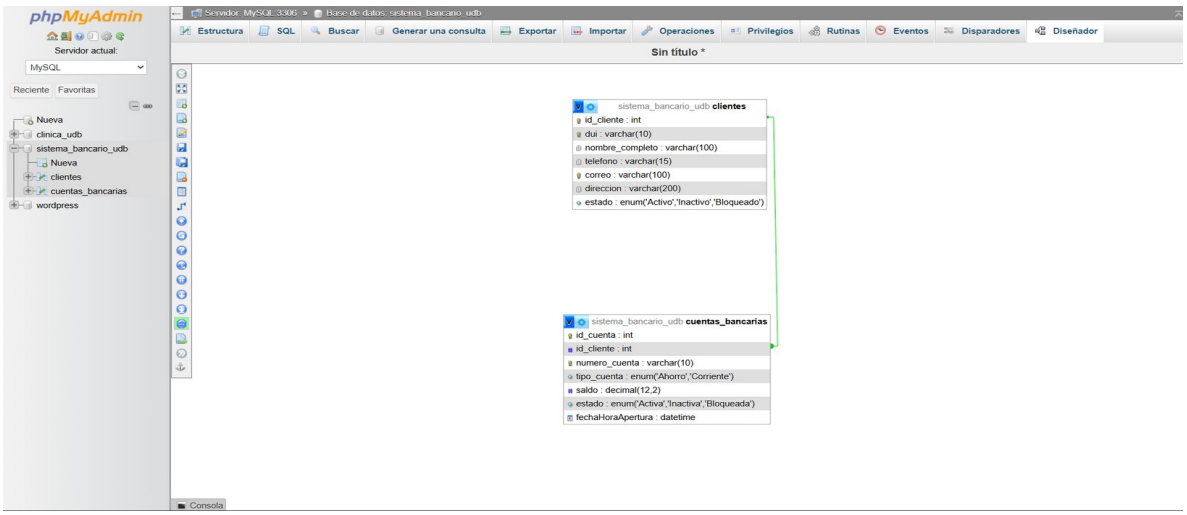
Nota. Elaboración propia mediante phpMyAdmin.

8.3. Relación entre las tablas

La base de datos utiliza una relación de tipo uno a muchos (1) entre las tablas clientes y cuentas_bancarias.

Un cliente puede tener una o varias cuentas bancarias, mientras que cada cuenta bancaria está asociada a un único cliente. Esta relación se establece mediante el campo id_cliente, que funciona como clave primaria en la tabla clientes y como clave foránea en cuentas_bancarias.

Figura 4. *Modelo relacional de la base de datos sistema_bancario_udb.*



Nota. Elaboración propia mediante la herramienta Designer de phpMyAdmin.

8.4. Restricciones de integridad referencial

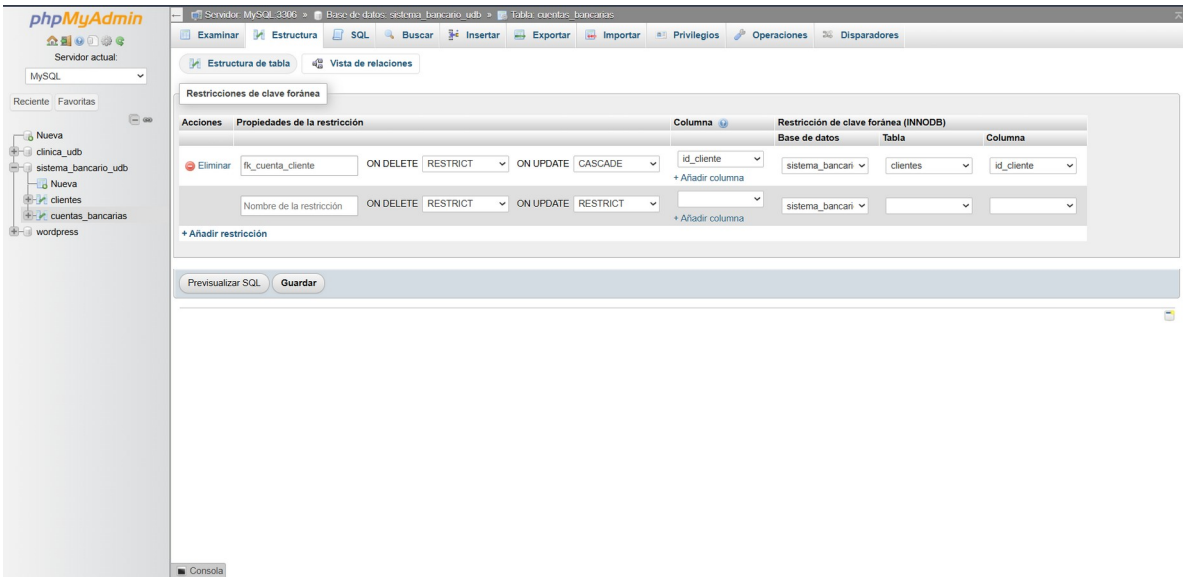
Para garantizar la integridad de los datos se implementó la restricción de clave foránea denominada fk_cuenta_cliente, que vincula el campo cuentas_bancarias.id_cliente con clientes.id_cliente.

La restricción utiliza las siguientes acciones:

- **ON DELETE RESTRICT:** impide eliminar un cliente que tenga cuentas bancarias asociadas, evitando registros relacionados sin propietario.
- **ON UPDATE CASCADE:** permite que los cambios realizados en el identificador del cliente se propaguen a los registros relacionados de la tabla de cuentas bancarias.

Además, se establecieron claves primarias, restricciones UNIQUE, campos obligatorios mediante NOT NULL y valores predeterminados para mantener la consistencia de la información almacenada.

Figura 5. *Configuración de la clave foránea fk_cuenta_cliente y sus restricciones de integridad referencial.*



Nota. Elaboración propia mediante phpMyAdmin.

8.5. Herramienta de administración de la base de datos

Para administrar y verificar la estructura de la base de datos se utilizó phpMyAdmin, herramienta que permitió consultar las tablas, revisar sus campos, comprobar las claves primarias y foráneas, y visualizar gráficamente el modelo relacional.

Esta herramienta facilitó la comprobación de la estructura y las restricciones implementadas en MySQL durante el desarrollo del proyecto.

9. Conexión JDBC y configuración de acceso a MySQL

9.1. Tecnología de conexión

Para establecer la comunicación entre la aplicación de escritorio desarrollada en Java y la base de datos MySQL se utilizó **JDBC (Java Database Connectivity)**. Esta tecnología permite ejecutar instrucciones SQL desde Java y realizar operaciones de consulta, inserción, actualización y eliminación de registros.

La conexión se organizó mediante una clase independiente denominada Conexion.java, ubicada en el paquete conexion. Esta separación permite centralizar el acceso a la base de datos y reutilizar la conexión desde las clases encargadas de gestionar la información.

9.2. Configuración de la conexión

Los parámetros de conexión se almacenaron en un archivo externo denominado database.properties, ubicado dentro de la carpeta física config_database del proyecto.

Este archivo contiene la dirección del servidor MySQL, el nombre de la base de datos, el usuario y la contraseña necesarios para establecer la conexión.

La configuración utilizada durante el desarrollo fue:

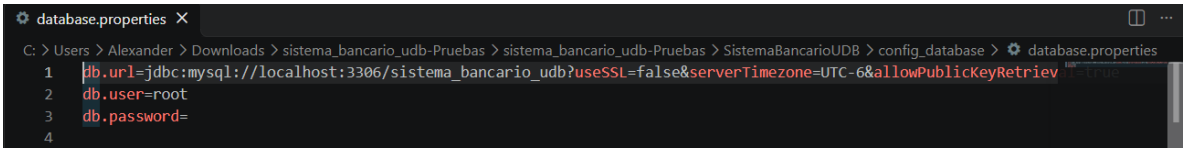
db.url=jdbc:mysql://localhost:3306/sistema_bancario_udb?
useSSL=false&serverTimezone=UTC-6&allowPublicKeyRetrieval=true

db.user=root

db.password=

El uso de un archivo de propiedades permite mantener los parámetros de conexión separados del código fuente, facilitando su modificación cuando sea necesario.

Figura 6. *Archivo de configuración de conexión a la base de datos.*



Nota. Elaboración propia.

9.3. Implementación de la conexión JDBC

Se implementó la clase Conexion.java, cuya responsabilidad es cargar los parámetros definidos en database.properties y proporcionar el método obtenerConexion(), encargado de establecer y devolver una conexión con MySQL mediante DriverManager.

Para la lectura del archivo se utiliza un bloque try-with-resources, que permite cerrar automáticamente el flujo de lectura una vez finalizado su uso. Asimismo, se contempla el manejo de excepciones relacionadas con la configuración y la conexión.

De esta manera, las demás clases del proyecto pueden solicitar una conexión sin tener que repetir el procedimiento de lectura de configuración.

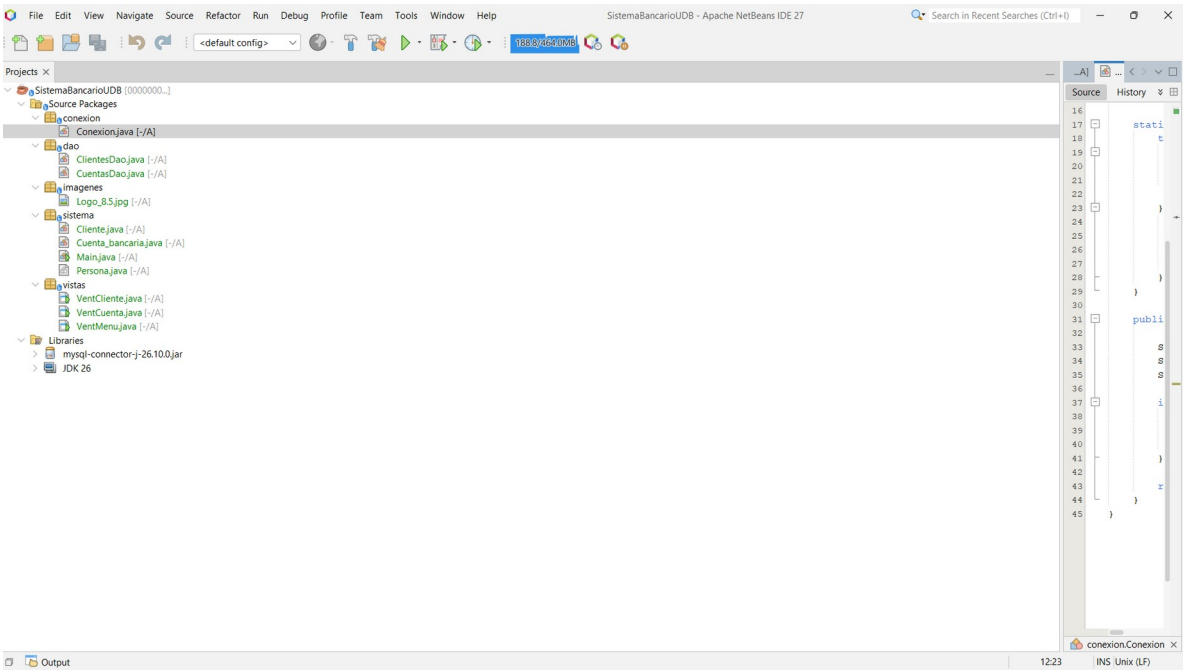
9.4. Organización de la capa de acceso a datos

Para separar las operaciones de persistencia de la interfaz gráfica se implementó el patrón DAO (Data Access Object). Esta organización permite concentrar las consultas SQL y las operaciones de acceso a datos en clases específicas.

El proyecto cuenta con los siguientes componentes:

- **Paquete conexion:** contiene la clase Conexion.java, encargada de gestionar la conexión con MySQL.
- **Paquete dao:** contiene las clases ClientesDao.java y CuentasDao.java, responsables de realizar las operaciones de acceso y manipulación de datos correspondientes a clientes y cuentas bancarias.

Figura 7. *Organización de los componentes de conexión y acceso a datos del proyecto.*



Nota. Elaboración propia.

9.5. Operaciones CRUD y manejo de recursos

Para interactuar con MySQL se utilizaron objetos PreparedStatement, que permiten preparar y ejecutar instrucciones SQL mediante parámetros.

También se implementó try-with-resources para gestionar automáticamente el cierre de las conexiones y sentencias utilizadas en las operaciones de acceso a datos.

Las operaciones CRUD desarrolladas permiten:

- **Create (Crear):** registrar nuevos clientes y cuentas bancarias.
- **Read (Consultar):** obtener los registros almacenados en la base de datos.
- **Update (Actualizar):** modificar la información de clientes y cuentas bancarias.
- **Delete (Eliminar):** eliminar registros respetando las restricciones de integridad referencial establecidas en MySQL.

Las operaciones que interactúan con la base de datos contemplan el manejo de excepciones SQLException, permitiendo identificar y comunicar errores durante la ejecución de las consultas.

Esta implementación permite mantener la persistencia de la información y facilita la comunicación entre la interfaz gráfica y la base de datos.

10. Implementación de la interfaz gráfica de usuario

10.1. Diseño de la interfaz gráfica

La interfaz gráfica del Sistema de Gestión Bancaria Académica fue desarrollada utilizando Java Swing, una biblioteca de Java que permite construir aplicaciones de escritorio mediante componentes visuales. Su implementación permite que el usuario interactúe con las funcionalidades del sistema a través de ventanas, botones, campos de texto, tablas y otros elementos gráficos.

El diseño se organizó en ventanas independientes para facilitar el acceso a las operaciones de gestión de clientes y cuentas bancarias. Se procuró mantener una distribución clara de los componentes, de manera que el usuario pueda ingresar información, consultar registros y ejecutar las operaciones disponibles de forma sencilla.

Asimismo, la interfaz se integró con las capas de acceso a datos mediante las clases DAO y la conexión JDBC, permitiendo que las operaciones realizadas desde las ventanas se reflejen en la base de datos MySQL.

Esta separación entre la interfaz gráfica, la lógica de acceso a datos y la base de datos contribuye a mantener una estructura organizada y facilita el mantenimiento del proyecto.

10.2. Ventana principal y navegación

La ventana principal del sistema constituye el punto de acceso a las funcionalidades de la aplicación. Fue desarrollada mediante Java Swing y permite al usuario navegar entre los módulos disponibles a través de botones que facilitan el acceso a las diferentes opciones.

Desde esta ventana se puede acceder a los módulos de gestión de clientes y cuentas bancarias. Cada opción abre su respectiva ventana, permitiendo realizar las operaciones correspondientes sin necesidad de interactuar directamente con la base de datos.

La navegación se organizó de manera sencilla para que el usuario pueda identificar las opciones disponibles y acceder a ellas de forma directa. Además, la aplicación utiliza una clase principal que inicia la interfaz gráfica mediante SwingUtilities.invokeLater, asegurando que la ventana se ejecute en el hilo de eventos de Swing.

Figura 8. *Ventana principal del Sistema de Gestión Bancaria Académica*

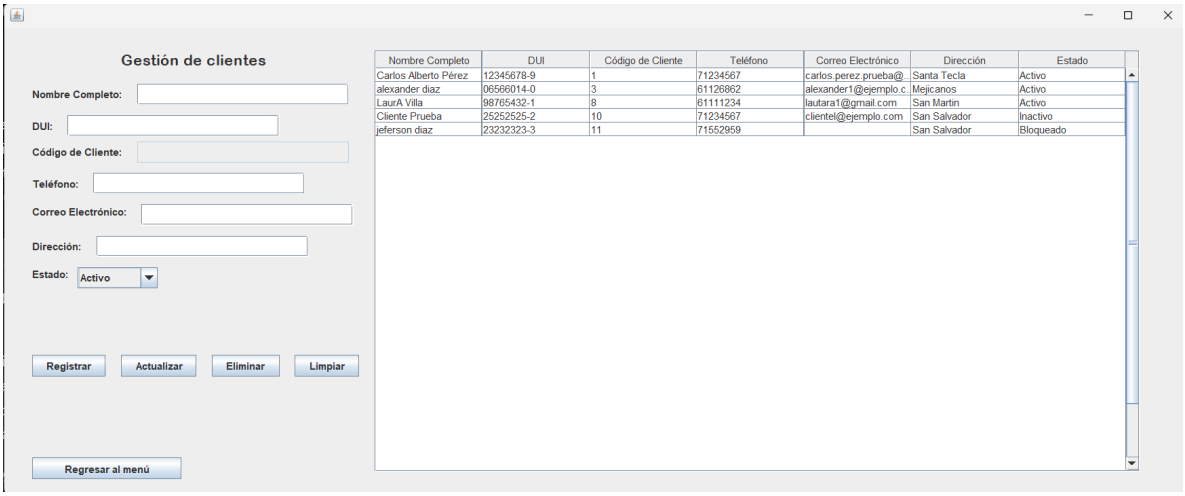


Nota. Interfaz principal desde la cual se accede a los módulos de gestión de clientes y cuentas bancarias.

10.3. Módulo de gestión de clientes

El módulo de gestión de clientes permite administrar la información personal de los clientes registrados en el Sistema de Gestión Bancaria Académica. Su interfaz, desarrollada mediante Java Swing, dispone de campos para ingresar los datos correspondientes al DUI, nombre completo, teléfono, correo electrónico y dirección, así como una tabla para visualizar los registros almacenados.

Figura 9. *Interfaz del módulo de gestión de clientes*



Nota. Interfaz gráfica utilizada para la administración de los registros de clientes.

El módulo implementa las operaciones CRUD (crear, consultar, actualizar y eliminar), permitiendo registrar nuevos clientes, visualizar la información existente, modificar sus datos y eliminar registros cuando las restricciones de integridad referencial de la base de datos lo permiten.

Para garantizar la consistencia de la información ingresada, se incorporaron validaciones en los formularios. Entre ellas se encuentra la comprobación de campos obligatorios, la verificación del formato del DUI mediante una expresión regular y la validación de la estructura del correo electrónico cuando este es proporcionado. El correo electrónico se considera un campo opcional, por lo que el sistema permite registrar clientes sin especificarlo.

Las operaciones de acceso y modificación de los registros se realizan mediante la clase ClientesDao, que utiliza JDBC y sentencias preparadas (PreparedStatement) para ejecutar las consultas correspondientes en MySQL. Asimismo, se emplea el manejo de excepciones SQL para atender posibles errores durante las operaciones de persistencia.

10.4. Módulo de gestión de cuentas bancarias

El módulo de gestión de cuentas bancarias permite administrar las cuentas asociadas a los clientes registrados en el sistema. Su interfaz gráfica, desarrollada mediante Java Swing, facilita el registro, consulta, actualización y eliminación de cuentas, mostrando información como el número de cuenta, DUI del cliente, nombre del titular, tipo de cuenta, saldo, estado y fecha de apertura.

Para registrar una cuenta, el sistema solicita el DUI del cliente y verifica que este se encuentre registrado y cumpla con las condiciones establecidas para la creación de cuentas. El número de cuenta se genera automáticamente, evitando que el usuario tenga que introducirlo manualmente. Asimismo, se utilizan tipos de datos adecuados para el manejo de valores monetarios, como BigDecimal, con el propósito de mantener la precisión de los saldos.

El módulo también incorpora reglas de negocio relacionadas con el estado del cliente y de la cuenta. Por ejemplo, los clientes bloqueados no pueden crear ni modificar cuentas, mientras que los clientes inactivos tienen restricciones para crear nuevas cuentas. De igual manera, las cuentas inactivas o bloqueadas no permiten modificar su saldo o tipo de cuenta, aunque sí pueden realizarse cambios en su estado bajo las condiciones establecidas.

Adicionalmente, se protege la relación entre el cliente y su cuenta, impidiendo modificar el propietario original de una cuenta después de su creación. Esta regla se respalda en la operación de actualización de la clase CuentasDao, que modifica únicamente el tipo de cuenta, el saldo y el estado, sin alterar la clave foránea que identifica al cliente propietario.

Las operaciones de persistencia se gestionan mediante la clase CuentasDao, encargada de ejecutar las consultas SQL utilizando JDBC y sentencias preparadas (PreparedStatement). De esta manera, el módulo mantiene la relación entre los clientes y sus cuentas, respetando las restricciones de integridad referencial definidas en la base de datos MySQL.

11. Validaciones y reglas de negocio

11.1. Validaciones de los datos de clientes

El módulo de gestión de clientes incorpora validaciones para comprobar que la información ingresada cumpla con los requisitos establecidos antes de almacenarse en la base de datos. Estas comprobaciones permiten reducir el ingreso de datos incompletos o con formatos incorrectos.

Entre las validaciones implementadas se encuentran:

- **Campos obligatorios:** se comprueba que el DUI, nombre completo, teléfono y dirección contengan información antes de registrar o actualizar un cliente.
- **Formato del DUI:** se verifica que el documento de identidad cumpla con la estructura de ocho dígitos, un guion y un dígito adicional (XXXXXXXX-X).
- **Correo electrónico:** cuando el usuario proporciona una dirección de correo, se comprueba que tenga una estructura válida. Este campo es opcional, por lo que puede dejarse vacío.
- **Integridad de los datos:** la base de datos establece restricciones de unicidad para el DUI y el correo electrónico, evitando que se registren valores duplicados.

Estas validaciones se aplican tanto al registrar como al actualizar clientes. Cuando la información no cumple con las condiciones establecidas, el sistema impide continuar con la operación y comunica al usuario el problema detectado.

11.2. Validaciones de cuentas bancarias

El módulo de gestión de cuentas bancarias incorpora validaciones para garantizar que la información ingresada cumpla con las condiciones establecidas por el sistema antes de realizar operaciones de registro o actualización.

Entre las validaciones implementadas se encuentran:

- **Identificación del cliente:** se comprueba que el DUI ingresado corresponda a un cliente existente en la base de datos antes de permitir la creación de una cuenta.
- **Saldo inicial:** se verifica que el saldo ingresado sea un valor numérico válido y que no sea negativo.
- **Número de cuenta:** este dato se genera automáticamente mediante el sistema, por lo que el usuario no necesita ingresarlo manualmente.
- **Tipo de cuenta:** se limita la selección a los tipos de cuenta disponibles: ahorro y corriente.
- **Estado de la cuenta:** se controla que el estado corresponda a las opciones establecidas por el sistema: activa, inactiva o bloqueada.

Adicionalmente, durante la actualización de una cuenta se aplican restricciones para evitar modificaciones que no estén permitidas según su estado. Estas comprobaciones contribuyen a mantener la consistencia de la información y a prevenir operaciones incompatibles con las condiciones de funcionamiento del sistema.

11.3. Reglas de negocio y restricciones

El sistema incorpora reglas de negocio que regulan las operaciones permitidas sobre los clientes y sus cuentas bancarias, con el propósito de mantener la integridad y consistencia de la información.

Las principales reglas implementadas son las siguientes:

- **Estado del cliente:** los clientes activos pueden crear y modificar cuentas bancarias. Los clientes inactivos no pueden crear nuevas cuentas, aunque pueden modificar cuentas existentes siempre que estas se encuentren activas. Los clientes bloqueados no pueden crear ni modificar cuentas.
- **Estado de la cuenta:** las cuentas activas permiten modificar su tipo y saldo, además de actualizar su estado. En cambio, cuando una cuenta se encuentra inactiva o bloqueada, únicamente se permite modificar su estado.
- **Conservación del propietario:** una cuenta bancaria debe conservar el cliente al que fue asociada durante su creación. El sistema impide cambiar el propietario de una cuenta mediante las operaciones de actualización.
- **Integridad referencial:** la base de datos mantiene una relación entre clientes y cuentas bancarias mediante una clave foránea. Esta restricción impide eliminar un cliente que todavía tenga cuentas asociadas.

Estas reglas se aplican para controlar las operaciones disponibles en el módulo de cuentas y proteger la relación entre los registros almacenados. Su cumplimiento permite evitar modificaciones no autorizadas por las condiciones establecidas para el funcionamiento del sistema.

11.4 Manejo de errores y persistencia de datos

El acceso a la base de datos se realiza mediante las clases DAO, las cuales utilizan PreparedStatement para ejecutar las consultas SQL y try-with-resources para gestionar automáticamente el cierre de las conexiones y demás recursos utilizados. Asimismo, las operaciones de acceso a datos contemplan el manejo de excepciones SQLException, permitiendo identificar errores relacionados con la ejecución de consultas y las restricciones de la base de datos.

Entre las situaciones que pueden generar errores se encuentran el registro de datos duplicados, la búsqueda de clientes inexistentes y los intentos de eliminar clientes que poseen cuentas bancarias asociadas. En este último caso, la restricción de clave foránea impide eliminar un cliente mientras existan cuentas vinculadas a él.

La persistencia de la información se realiza mediante MySQL. Los registros almacenados pueden consultarse nuevamente al cerrar y abrir la aplicación, sin necesidad de registrarlos otra vez.

12. Casos de prueba

12.1. Objetivo y alcance de las pruebas

Los casos de prueba se elaboraron con el propósito de verificar el funcionamiento de las principales operaciones del Sistema de Gestión Bancaria Académica, así como comprobar el cumplimiento de las validaciones y reglas de negocio implementadas.

Las pruebas abarcan los módulos de gestión de clientes y cuentas bancarias, considerando operaciones de registro, consulta, actualización y eliminación. También se contemplan escenarios con información inválida, restricciones relacionadas con los estados de los clientes y las cuentas, integridad referencial y persistencia de los datos.

Para cada caso se considera la operación evaluada, los datos de entrada, el resultado esperado y el resultado obtenido. Esto permite identificar posibles errores, comprobar el comportamiento del sistema y documentar las evidencias de las pruebas realizadas.

12.2 Pruebas de clientes

En esta sección se presentan las pruebas realizadas al módulo de gestión de clientes, con el propósito de verificar el registro, actualización y eliminación de información, así como el cumplimiento de las validaciones establecidas. Se consideran tanto entradas válidas como inválidas para comprobar que el sistema permita las operaciones correctas y rechace los datos que no cumplen con las condiciones definidas.

Código	Prueba realizada	Datos de entrada o condiciones iniciales	Resultado esperado	Resultado obtenido	Estado
CP-01	Registro de cliente válido	DUI, nombre completo, teléfono, dirección y correo con formato válido.	Registrar correctamente al cliente en la base de datos.	El cliente se registró correctamente.	Aprobado

CP-02	Campos obligatorios vacíos	Intentar registrar un cliente dejando uno o más campos obligatorios vacíos.	Impedir el registro y solicitar completar los campos requeridos.	El sistema impidió guardar el registro.	Aprobado
CP-03	DUI con formato incorrecto	Ingresar un DUI que no cumpla con el formato establecido.	Rechazar el DUI y evitar el registro.	Después de corregir la validación, el sistema rechazó el formato incorrecto.	Aprobado
CP-04	Correo electrónico con formato incorrecto	Ingresar un correo electrónico con una estructura inválida.	Rechazar el correo electrónico y evitar el registro.	Después de corregir la validación, el sistema rechazó el formato incorrecto.	Aprobado
CP-05	Registro de DUI duplicado	Intentar registrar un cliente utilizando un DUI que ya existe.	Impedir el registro de un DUI duplicado.	La base de datos rechazó el registro por duplicidad.	Aprobado
CP-06	Registro sin correo electrónico	Dejar vacío el campo de correo electrónico e ingresar los demás campos obligatorios correctamente.	Permitir el registro, debido a que el correo electrónico es opcional.	El cliente se registró sin correo electrónico.	Aprobado
CP-07	Registro de correo electrónico duplicado	Intentar registrar un correo electrónico que ya está asociado a otro cliente.	Impedir el registro de un correo electrónico duplicado.	La base de datos rechazó el registro por duplicidad.	Aprobado
CP-08	Persistencia de clientes	Registrar un cliente, cerrar el módulo y volver a consultarlo.	Mantener la información almacenada en la base de datos.	El cliente continuó disponible al volver a consultar los registros.	Aprobado
CP-09	Actualización de información del cliente	Seleccionar un cliente existente y modificar su dirección.	Guardar los cambios y mostrar la información actualizada.	La dirección del cliente se actualizó correctamente.	Aprobado
CP-10	Eliminación de cliente sin cuentas asociadas	Seleccionar un cliente de prueba que no tenga cuentas bancarias vinculadas e intentar eliminarlo.	Permitir la eliminación del cliente.	El cliente se eliminó correctamente.	Aprobado

Los casos CP-03 y CP-04 permitieron identificar deficiencias iniciales en las validaciones del formulario. Estas fueron corregidas y posteriormente se repitieron las pruebas para comprobar el comportamiento esperado.

12.3 Pruebas de cuentas bancarias

En esta sección se presentan las pruebas realizadas al módulo de gestión de cuentas bancarias, con el propósito de verificar el registro, consulta, actualización y eliminación de cuentas. También se comprueba el cumplimiento de las validaciones relacionadas con el saldo, la identificación del cliente y la integridad de los datos almacenados en la base de datos.

Código	Prueba realizada	Datos de entrada o condiciones iniciales	Resultado esperado	Resultado obtenido	Estado
CP-11	Registro de cuenta para cliente activo	Ingresar el DUI de un cliente existente con estado Activo y completar los datos de la cuenta.	Permitir la creación de la cuenta y asociarla al cliente seleccionado.	La cuenta se registró correctamente y quedó asociada al cliente.	Aprobado
CP-12	Registro con DUI de cliente inexistente	Ingresar un DUI que no se encuentre registrado en la base de datos.	Impedir la creación de la cuenta por no existir el cliente.	El sistema impidió registrar la cuenta.	Aprobado
CP-13	Registro de saldo negativo	Intentar registrar una cuenta con un saldo menor que cero.	Rechazar el saldo negativo.	El sistema rechazó el valor ingresado.	Aprobado
CP-14	Registro de saldo no numérico	Ingresar letras u otros caracteres no numéricos en el campo de saldo.	Impedir el registro de un saldo que no sea numérico.	El sistema rechazó el valor ingresado.	Aprobado
CP-15	Generación automática del número de cuenta	Abrir el formulario de registro de una cuenta bancaria.	Generar automáticamente el número de cuenta sin permitir su edición manual.	El número de cuenta se generó automáticamente y no pudo modificarse desde el formulario.	Aprobado
CP-16	Registro de cuenta corriente con saldo cero	Seleccionar el tipo de cuenta Corriente e ingresar un saldo de 0.00.	Permitir el registro de la cuenta con saldo inicial cero.	La cuenta se registró correctamente con saldo cero.	Aprobado
CP-17	Actualización del saldo de una cuenta	Seleccionar una cuenta con saldo inicial de 0.00 e ingresar un nuevo saldo de	Actualizar el saldo y conservar los demás datos de la cuenta.	El saldo se actualizó correctamente a 150.50.	Aprobado

		150.50.			
CP-18	Eliminación de cuenta bancaria	Seleccionar una cuenta existente e intentar eliminarla.	Eliminar la cuenta seleccionada de la base de datos.	La cuenta se eliminó correctamente.	Aprobado
CP-19	Eliminación de cliente con cuenta asociada	Seleccionar un cliente que tenga al menos una cuenta bancaria vinculada e intentar eliminarlo.	Impedir la eliminación para conservar la integridad referencial.	La restricción de clave foránea impidió eliminar el cliente.	Aprobado
CP-20	Persistencia de cuentas bancarias	Registrar una cuenta, cerrar el módulo y volver a consultar los registros.	Mantener la información de la cuenta almacenada en la base de datos.	La cuenta continuó disponible al volver a consultar los registros.	Aprobado

12.4 Pruebas de estados de clientes y cuentas bancarias

En esta sección se documentan las pruebas realizadas para verificar las restricciones establecidas según el estado de los clientes y de sus cuentas bancarias. Se comprueba que el sistema permita o impida determinadas operaciones de acuerdo con las reglas de negocio implementadas.

Código	Prueba realizada	Datos de entrada o condiciones iniciales	Resultado esperado	Resultado obtenido	Estado
CP-21	Registro de cuenta para cliente inactivo	Seleccionar un cliente con estado Inactivo e intentar crear una cuenta bancaria.	Impedir la creación de cuentas para clientes inactivos.	El sistema impidió registrar la cuenta.	Aprobado
CP-22	Registro de cuenta para cliente bloqueado	Seleccionar un cliente con estado Bloqueado e intentar crear una cuenta bancaria.	Impedir la creación de cuentas para clientes bloqueados.	El sistema impidió registrar la cuenta.	Aprobado
CP-23	Modificación de cuenta activa de cliente inactivo	Seleccionar una cuenta en estado Activa cuyo propietario tenga estado Inactivo e intentar modificar sus datos.	Permitir la modificación de la cuenta activa, respetando las restricciones del sistema.	El sistema permitió modificar la cuenta.	Aprobado
CP-24	Modificación de cuenta de cliente bloqueado	Seleccionar una cuenta perteneciente a un cliente con estado Bloqueado e intentar modificarla.	Impedir la modificación de la cuenta.	El sistema bloqueó la operación.	Aprobado

CP-25	Modificación de saldo de cuenta inactiva	Seleccionar una cuenta con estado Inactiva e intentar modificar su saldo.	Impedir la modificación del saldo mientras la cuenta permanezca inactiva.	El sistema impidió modificar el saldo.	Aprobado
CP-26	Reactivación de cuenta inactiva	Seleccionar una cuenta Inactiva y cambiar únicamente su estado a Activa.	Permitir la reactivación sin modificar otros datos de la cuenta.	La cuenta se reactivó correctamente y conservó sus demás datos.	Aprobado
CP-27	Bloqueo de cuenta activa	Seleccionar una cuenta Activa y cambiar su estado a Bloqueada.	Permitir cambiar el estado de la cuenta a Bloqueada.	El sistema permitió bloquear la cuenta.	Aprobado
CP-28	Modificación de saldo de cuenta bloqueada	Seleccionar una cuenta Bloqueada e intentar modificar su saldo.	Impedir la modificación del saldo de una cuenta bloqueada.	El sistema impidió modificar el saldo.	Aprobado
CP-29	Reactivación de cuenta bloqueada	Seleccionar una cuenta Bloqueada y cambiar únicamente su estado a Activa.	Permitir la reactivación sin modificar otros datos de la cuenta.	La cuenta se reactivó correctamente y conservó sus demás datos.	Aprobado
CP-30	Cambio de propietario de una cuenta	Seleccionar una cuenta existente e intentar modificar el cliente al que está asociada.	Impedir el cambio de propietario de una cuenta bancaria.	El sistema mantuvo la asociación original de la cuenta.	Aprobado

13. Evidencias de ejecución

13.1. Evidencias de la interfaz gráfica

En este apartado se presentan capturas de pantalla que permiten visualizar la ejecución de los principales componentes del Sistema de Gestión Bancaria Académica. Estas evidencias respaldan la implementación de la interfaz gráfica y el acceso a los módulos desarrollados.

Figura 10. *Ventana principal del Sistema de Gestión Bancaria Académica*

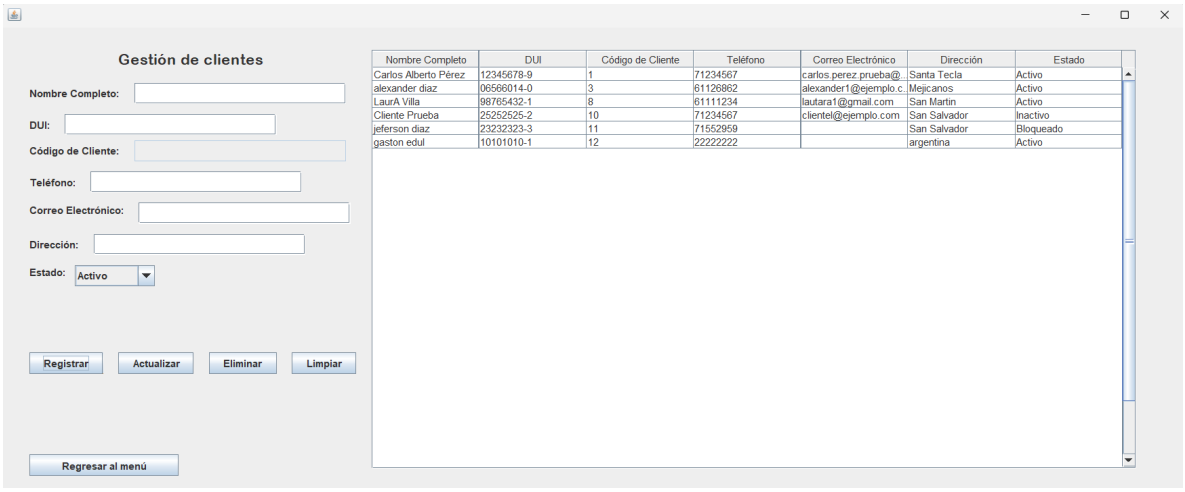


Nota. Captura de pantalla obtenida durante la ejecución del Sistema de Gestión Bancaria Académica como evidencia de su funcionamiento.

13.2. Evidencias de gestión de clientes y cuentas bancarias

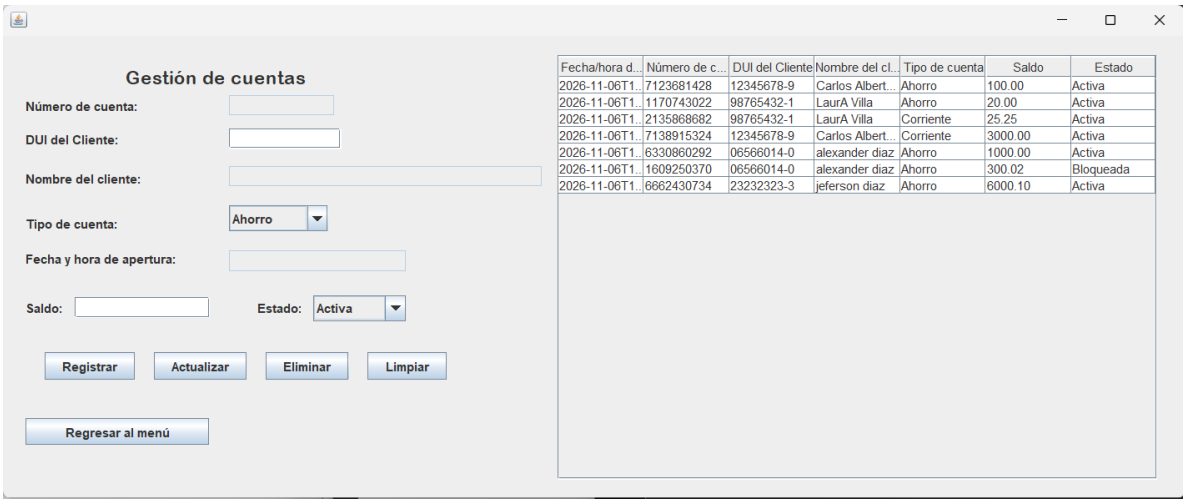
Las siguientes evidencias muestran los módulos principales de administración de clientes y cuentas bancarias, permitiendo identificar los formularios y la información presentada por el sistema.

Figura 11. *Módulo de gestión de clientes*



Nota. Captura de pantalla obtenida durante la ejecución del Sistema de Gestión Bancaria Académica como evidencia de su funcionamiento.

Figura 12. *Módulo de gestión de cuentas bancarias*

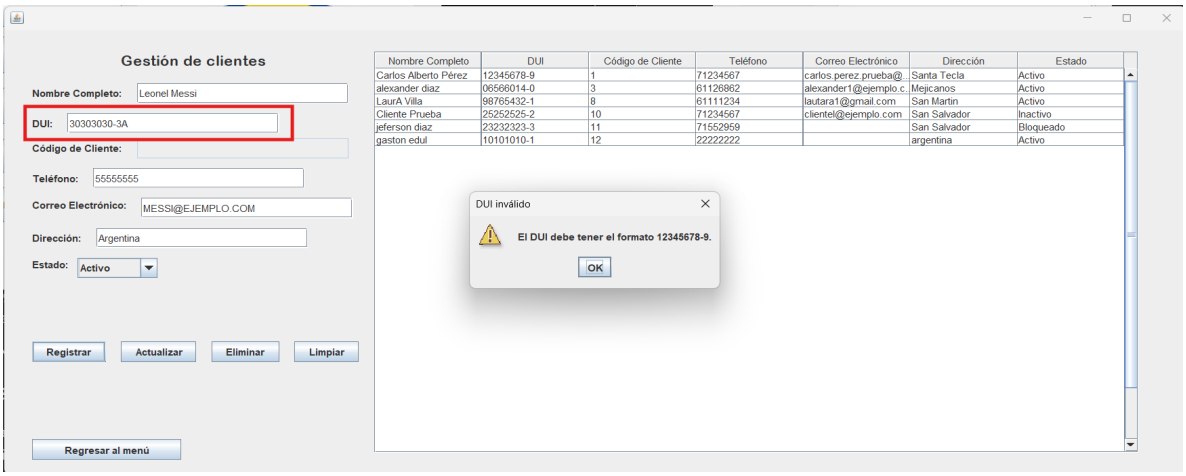


Nota. Captura de pantalla obtenida durante la ejecución del Sistema de Gestión Bancaria Académica como evidencia de su funcionamiento.

13.3. Evidencias de validaciones e integridad de datos

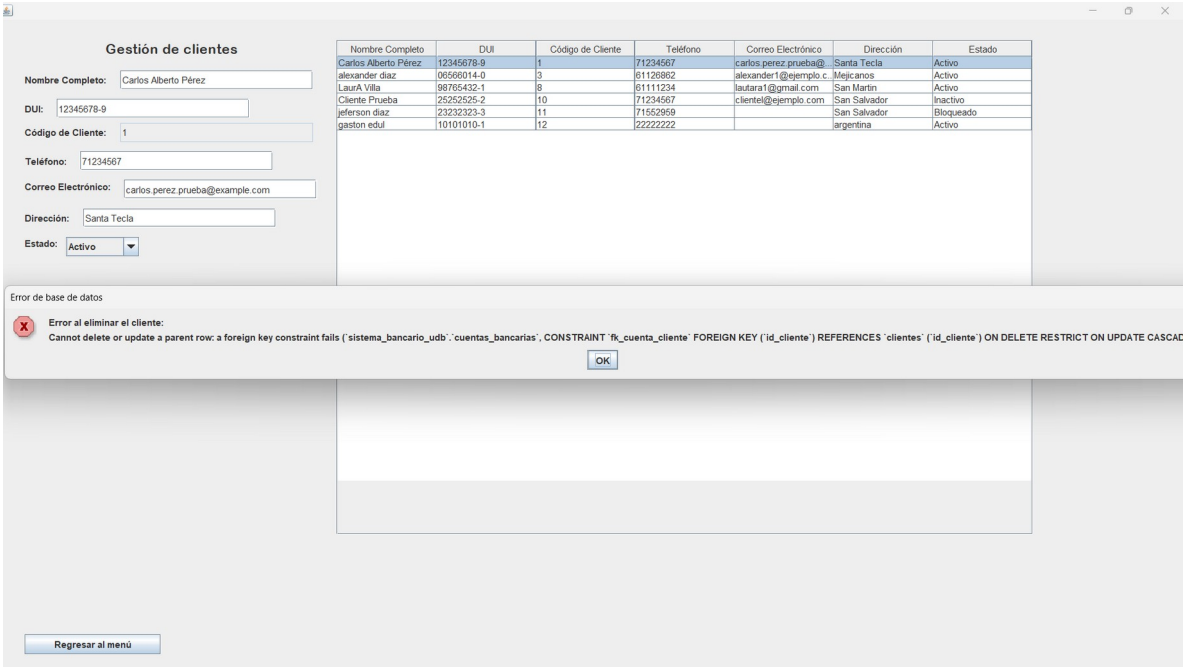
Se presentan evidencias seleccionadas del comportamiento del sistema ante situaciones que requieren controles de validación y protección de la información almacenada.

Figura 13. *Validación de los datos ingresados*



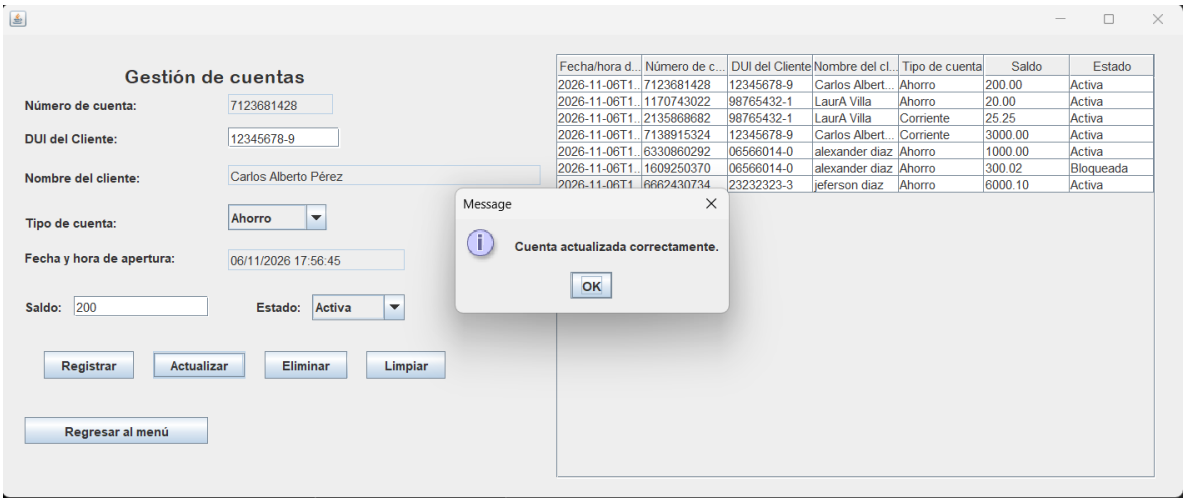
Nota. Captura de pantalla obtenida durante la ejecución del Sistema de Gestión Bancaria Académica como evidencia de su funcionamiento.

Figura 14. *Restricción de integridad referencial*



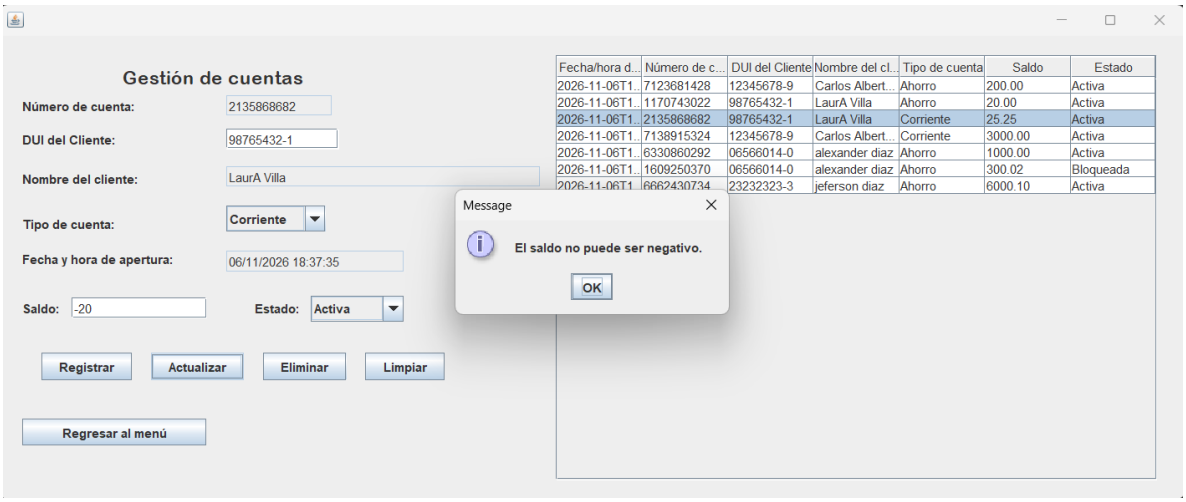
Nota. Captura de pantalla obtenida durante la ejecución del Sistema de Gestión Bancaria Académica como evidencia de su funcionamiento.

Figura 15. *Actualización de cuenta bancaria*



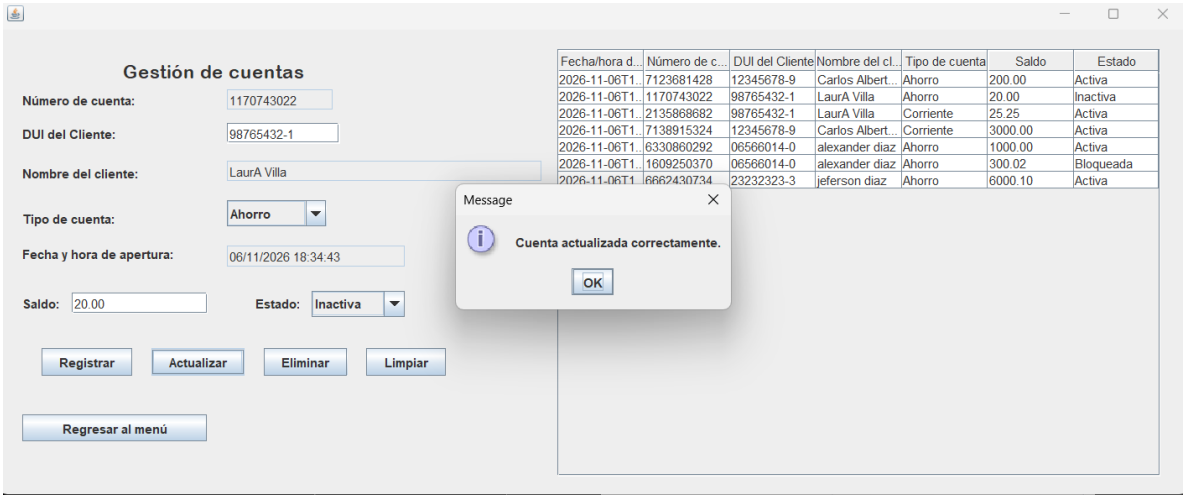
Nota. Captura de pantalla obtenida durante la ejecución del Sistema de Gestión Bancaria Académica como evidencia de su funcionamiento.

Figura 16. *Validación de saldo de cuenta*



Nota. Captura de pantalla obtenida durante la ejecución del Sistema de Gestión Bancaria Académica como evidencia de su funcionamiento.

Figura 17. *Gestión de estados de cuentas bancarias*



Nota. Captura de pantalla obtenida durante la ejecución del Sistema de Gestión Bancaria Académica como evidencia de su funcionamiento.

14. Conclusiones

1. El desarrollo del Sistema de Gestión Bancaria Académica permitió aplicar los principios de la programación orientada a objetos mediante la implementación de clases, atributos encapsulados, herencia y relaciones entre entidades, facilitando la organización y mantenimiento del código.
2. La implementación de una base de datos en MySQL y la conexión mediante JDBC permitió almacenar y gestionar la información de clientes y cuentas bancarias de forma persistente, utilizando operaciones CRUD para registrar, consultar, actualizar y eliminar información.
3. El desarrollo de interfaces gráficas con Java Swing permitió disponer de una aplicación de escritorio para administrar los registros de clientes y cuentas bancarias, incorporando validaciones que contribuyen a reducir el ingreso de información incorrecta.
4. Las pruebas realizadas permitieron comprobar el funcionamiento de las operaciones implementadas, así como las restricciones relacionadas con los datos obligatorios, formatos, saldos, estados de clientes y cuentas, y la integridad referencial de la base de datos.
5. El trabajo realizado permitió fortalecer los conocimientos prácticos sobre programación orientada a objetos, gestión de bases de datos, conexión mediante JDBC y desarrollo de aplicaciones de escritorio, estableciendo una base para futuras ampliaciones del sistema.

Referencias.

MySQL. (2024). *MySQL Connector/J developer guide*. Oracle. https://dev.mysql.com/doc/connector-j/en/

MySQL. (2024). *MySQL 8.0 reference manual*. Oracle. https://dev.mysql.com/doc/refman/8.0/en/

Oracle. (s. f.-a). *Creating a GUI with Swing*. Java Documentation. https://docs.oracle.com/javase/tutorial/uiswing/

Oracle. (s. f.-b). *JDBC overview*. Java Documentation. https://docs.oracle.com/javase/8/docs/technotes/guides/jdbc/

Oracle. (2023). *Java Platform, Standard Edition & Java Development Kit specifications*. https://docs.oracle.com/en/java/javase/